数据分析与算法实践课程 · 实验报告（选题二）

基于 MovieLens 的残差协同过滤电影推荐系统

紧凑索引、正则化双偏差基线与离线对照评估

一、小组基本信息与成员贡献表

本报告以小组为单位提交。发送前请核对或填入小组成员姓名与学号。

小组角色	姓名	学号	成员具体贡献备注
组长	_ _ _ _ _ _ _	_ _ _ _ _ _ _ _	负责实验总体方案、正则化双偏差与残差 UserCF/ItemCF 数学建模、时序验证集调参及报告统稿（贡献度 40%）。
组员 1	_ _ _ _ _ _ _	_ _ _ _ _ _ _ _	负责数据核对、紧凑索引与稀疏矩阵实现、阴性/阳性对照及配对 bootstrap 评估（贡献度 30%）。
组员 2	_ _ _ _ _ _ _	_ _ _ _ _ _ _ _	负责纯 Python 模块化、prompt_toolkit 全屏 TUI、图表绘制与复现脚本整理（贡献度 30%）。

二、实验选题与核心工作摘要

本组选择实验内容二：利用公开数据集设计基于协同过滤的电影推荐系统。实验使用 MovieLens ml-latest-small（100,004 条评分、671 位用户、9,066 部有评分电影），完成三项工作：

- **紧凑索引与残差协同建模：**用连续位置映射替代原始 `movieId` 下标，以 NaN 表示缺失评分并用布尔掩码隔离；先拟合正则化用户/电影双偏差基线 $b_{ui} = \mu + b_u + b_i$，再在残差上计算带共同评分门槛与收缩的余弦相似度，融合 UserCF 与 ItemCF 两条修正通道。
- **可复查的时序评估：**按每位用户的时间戳做外层 80%/20% 留出（训练 79,748 条，测试 20,256 条），并在训练集内部再留出验证集（16,209 条）调参。测试集 RMSE 从用户均分基线的 0.9750 降至双偏差的 0.9072，最终混合模型为 0.8979；MAE 从 0.7477 降至 0.6851。另报告随机邻居阴性对照、合成信号阳性对照、逐项消融和按用户配对 bootstrap 区间。
- **纯 Python 工程与终端界面：**实现 `recommender.py`、`tui.py`、`main.py` 三个模块；TUI 基于 prompt_toolkit，提供用户、流派子串和片名子串筛选，F2/F3 翻页，Tab 切换输入框，Ctrl-Q 退出，并直接读取离线 Top-50 推荐缓存。

课程提交核对项	具体指标与格式要求
正文字数与附录页数	正文按课程要求排版；附录为实际核心代码节选，控制在 3 页以内。
截止时间与接收邮箱	11 月 15 日 23:59 前发送至 `zhh_swufe@126.com`，邮件只需发送一次。
邮件主题与压缩包命名	邮件主题：周一12节实验报告：组长姓名、组员1姓名、组员2姓名……；文件夹与压缩包同名：周一12节：组长姓名、组员1姓名、组员2姓名……

1 实验背景、数据特征与实现风险

1.1 MovieLens 数据集结构与评分分布

个性化推荐需要从高度稀疏的历史交互中区分全局统计规律与个体偏好。本实验采用 GroupLens 发布的 MovieLens ml-latest-small 基准集：100,004 条带时间戳的显式评分，671 位用户（每人至少 20 条评分），9,066 部至少被评分一次的电影，电影目录共 9,125 部。每部电影由 `movieId` 唯一标识，`movies.csv` 提供英文片名、上映年份和竖线分隔的流派标签。数据中有 19 个命名流派（Action、Adventure、Animation、Children、Comedy、Crime、Documentary、Drama、Fantasy、Film-Noir、Horror、IMAX、Musical、Mystery、Romance、Sci-Fi、Thriller、War、Western），另有 18 部影片标记为 `(no genres listed)`。

评分采用 0.5 分至 5.0 分的半星十档制。全站平均分 $\mu = 3.5436$，标准差约 1.0581。不低于 3.0 分的正向评价共 82,170 条，占 82.17%；4.0 分是众数档位（28,750 条，占 28.75%），其次为 3.0 分（20,064 条，占 20.06%）和 5.0 分（15,095 条，占 15.09%）。2.0 分及以下共 13,385 条（13.38%），最低档 0.5 分有 1,101 条（1.10%）。高分聚集说明用户会主动选择预期感兴趣或口碑较好的影片，评分分布并非随机；因此模型应先估计非均匀的统计基线，再讨论个性化修正。

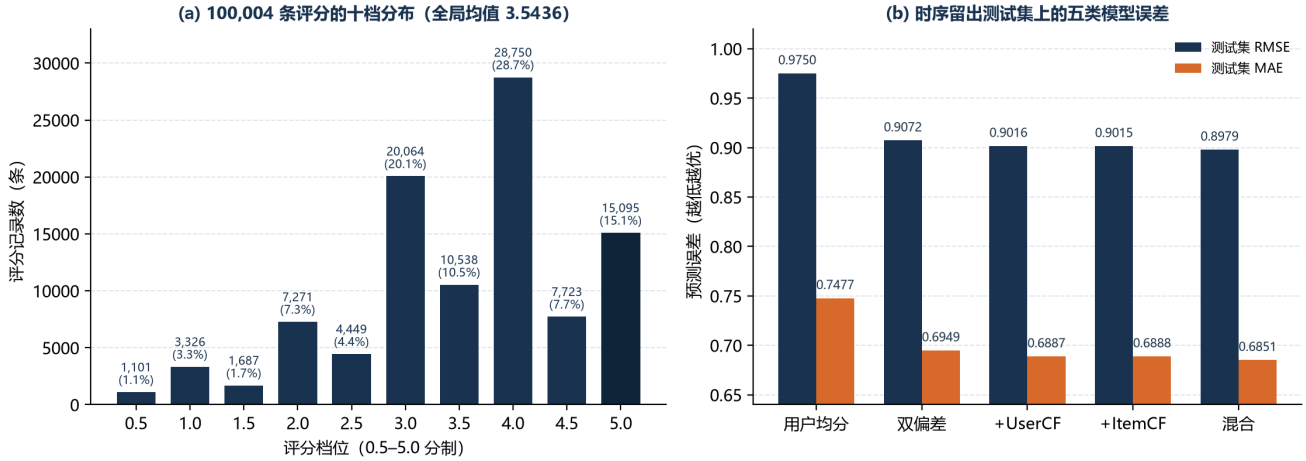


Figure 1: （a）MovieLens ml-latest-small 100,004 条评分的十档频数分布；（b）按用户时间顺序留出测试集（20,256 条）上五类模型的 RMSE 与 MAE。

1.2 原始 ID 直接建矩阵的四类风险

若沿用“把 `movieId` 直接当数组列下标”的直觉写法，会遇到以下四类问题。本项目并不依赖某个未随仓库提供的 Notebook，而是把这些问题作为实现约束逐项处理：

1. **离散 ID 作下标导致内存膨胀。**有评分的电影只有 9,066 部，但评分表中的最大 `movieId` 为 163,949。若直接构造 $(671, 163949)$ 的双精度矩阵，单个矩阵约 880.08 MB；其中 154,883 列（94.47%）从未出现评分。改为连续位置映射后，671×9066 的 float32 矩阵约 24.33 MB。
2. **缺失评分与真实低分混同。**若把评分缩放到 $[0, 1]$ 并把未评分位置填 0，0.5 分会与“缺失”得到相同的数值，相似度和加权预测会把未评分当成真实低分。本项目的评分矩阵用 `NaN` 表示缺失，并额外保存布尔观测掩码；残差矩阵只在矩阵乘法前把缺失位置置零，观测掩码负责隔离这些位置。
3. **预测矩阵切片轴向颠倒。**矩阵行轴对应用户、列轴对应电影。若写成 `user_prediction[:, userid]`，取到的是一部电影对所有用户的预测分，再把用户行号当作片名下标，推荐结果与目标用户无关。正确做法是沿用户行取 `pred[u_idx, :]`，过滤已评分电影后再用索引反查片名与流派。
4. **训练/测试独立建模导致评估错位。**若训练集和测试集各自建矩阵、各自训练，再跨矩阵比较预测值与测试评分，模型并没有在“只看过去”的条件下预测未来，指标不可解释。本项目使用按用户时间戳的嵌套切分：外层训练/测试，内层训练/验证；所有超参数只在内层验证集选择。

实现维度	直接使用原始 ID 的写法	本项目实现
索引与内存	以最大 `movieId`=163,949 作列下标，双精度矩阵约 880.08 MB。	双向连续映射 $0\dots670$ 与 $0\dots9065$；float32 密集矩阵约 24.33 MB，残差 CSR 约 0.80 MB，布尔掩码约 0.50 MB。
缺失值处理	缩放至 $[0,1]$ 后以 0 填充，最低分与缺失不可区分。	保留 0.5–5.0 原始量纲；`NaN` + 布尔掩码分离缺失与观测。
检索轴向	误用列切片，片名排序也未与 `movieId` 对齐。	沿用户行取预测，过滤已评分项，再用 `movieId` 反查元数据。
评估协议	训练集与测试集独立建模后比较，存在时间泄露与尺度错位。	按用户时间戳 80%/20% 留出；训练集内部再留 20% 验证集调参。

2 算法：从评分匹配到残差协同

2.1 传统 UserCF 的局限

传统基于用户的协同过滤寻找历史打分习惯相似的邻居，再用邻居对目标电影的评分偏离预测当前用户评分。令用户 u 的历史均分为 $\bar{r}_u$，用户 u 与 v 的相似度为 s_{uv}，经典中心化预测为：

$$\hat{r}_{ui}^{(\text{legacy})} = \bar{r}_u + \frac{\sum_{v\in N(u)} s_{uv}(r_{vi}-\bar{r}_v)}{\sum_{v\in N(u)}|s_{uv}|}.$$

减去用户均分只能消除部分个人打分尺度差异，电影本身的大众口碑仍会干扰相似度：两位用户都给经典高分电影高分、给口碑较差的影片低分，传统方法会认为二人相似，但这可能只是反映大众共识。原版算法在“比打分”，改进目标则是“比口味”。

2.2 第一阶段：正则化双偏差基线

为剥离非个性化统计主效应，先构建包含全局均值、用户宽严偏差和电影口碑偏差的双偏差基线：

$$b_{ui} = \mu + b_u + b_i.$$

其中 μ 为训练集已观测评分的全局均值；$b_u>0$ 表示用户相对大众打分偏松，$b_i>0$ 表示电影口碑偏高。为避免长尾电影和低频用户过拟合，引入 L_2 收缩项 λ_u,λ_i，交替更新 10 轮：

$$b_u \leftarrow \frac{\sum_{i\in I(u)}(r_{ui}-\mu-b_i)}{|I(u)|+\lambda_u}, \qquad b_i \leftarrow \frac{\sum_{u\in U(i)}(r_{ui}-\mu-b_u)}{|U(i)|+\lambda_i}.$$

本实验在验证集上选择 $\lambda_u=\lambda_i=5.0$。

2.3 第二阶段：残差、收缩相似度与阻尼修正

从训练集真实评分中扣除基础分，得到个体偏好残差：

$$e_{ui} = r_{ui} - b_{ui} = r_{ui} - (\mu + b_u + b_i), \quad (u,i)\in\Omega_{\text{train}}.$$

若一位平时打分严苛的用户给一部大众评分普通的冷门文艺片打出明显高于基础分的分数，就会产生正残差；若两位用户在多部电影上出现同向残差，说明二人可能有超越大众流行度的品味共鸣。只在共同评分交集 $I_{uv}=I(u)\cap I(v)$ 上计算残差余弦相似度，并加入最少共同评分门槛 $c_{\min}$ 与收缩系数 β_s：

$$s_{uv} = \mathbf{1}(|I_{uv}|\ge c_{\min})\cdot\frac{|I_{uv}|}{|I_{uv}|+\beta_s}\cdot\frac{\sum_{i\in I_{uv}} e_{ui}e_{vi}}{\sqrt{\sum_{i\in I_{uv}} e_{ui}^2}\sqrt{\sum_{i\in I_{uv}} e_{vi}^2}},$$

其中分母为 0 时相似度取 0。实现中先用 $C=MM^{\mathsf{T}}$ 统计共同评分数，再用零填充残差矩阵的 $E_{\text{dot}}=R_{\text{res}}R_{\text{res}}^{\mathsf{T}}$ 计算内积；缺失位置的残差为 0，不会贡献内积，交集由重叠计数和掩码保证。随后只保留正相似度，按行取 Top-K 邻居。

对目标电影 i，在给该片评分且与目标用户正相关的候选中取 Top-K 邻居集合 $N_K(u)$，用验证集选择的阻尼系数 α_u 控制近邻信号强度：

$$\Delta_u(u,i) = \frac{\sum_{v\in N_K(u)\cap U(i)} s_{uv}e_{vi}}{\sum_{v\in N_K(u)\cap U(i)} s_{uv}}, \qquad \hat{r}_{ui}^{(\text{UserCF})} = \text{clip}\left(b_{ui}+\alpha_u\Delta_u(u,i),\,0.5,\,5.0\right).$$

若 $b_{ui} = 3.80$，相似邻居的加权残差均值为 $+0.70$，未阻尼时会升到 4.50；采用 $\alpha_u = 0.2$ 后只吸纳 $0.2 \times 0.70 = +0.14$，得到 3.94 分。

2.4 ItemCF 与双通道混合

沿电影方向对称计算物品间收缩残差相似度 $s_{ij}^{(\text{item})}$。为避免 $9{,}066 \times 9{,}066$ 的稠密相似度矩阵，实现先用余弦最近邻为每部电影取 128 个候选，再只对候选对统计共同评分人数、做收缩并保留 Top-K。目标电影 i 的物品残差修正为：

$$\Delta_i(u,i) = \frac{\sum_{j \in N_K(i)} s_{ij}^{(\text{item})} e_{uj}}{\sum_{j \in N_K(i)} s_{ij}^{(\text{item})}}.$$

双通道融合后：

$$\hat{r}_{ui}^{(\text{Hybrid})} = \text{clip}\left(b_{ui} + \gamma\left(\alpha_u \Delta_u(u,i) + \alpha_i \Delta_i(u,i)\right), 0.5, 5.0\right).$$

表 1 列出验证集选择的参数。UserCF 与 ItemCF 的邻居数、共同评分门槛和阻尼强度分别调参，混合系数 γ 再在验证集上网格搜索。

Table 1: 验证集选择的模型参数

组件	参数	取值
双偏差	用户/电影正则化	$\lambda_u = \lambda_i = 5.0$（交替 10 轮）
UserCF	K / $c_{\min}$ / β_s / α_u	320 / 3 / 15.0 / 0.2
ItemCF	K / $c_{\min}$ / β_s / α_i	100 / 1 / 15.0 / 0.25
混合	混合系数 γ	0.75

3 评估协议与对照实验

3.1 时序切分与调参

推荐系统评估协议决定结论是否可信。Ferrari Dacrema 等人在 RecSys 2019 的复现研究指出，复杂模型不一定优于简单模型，为 UserKNN/ItemKNN 公平调参并配置强偏差基线十分重要[3]。Said 与 Bellogín 的复现综述、Said 等人对评估方法的再检讨也强调避免未来数据预测过去，并通过对照与消融验证组件增益[4,5]。

本项目按每位用户的 `timestamp`（同时间按 `movieId`）升序排序，取前 80% 作外层训练集（79,748 条），最近 20% 作测试集（20,256 条）；再对外层训练集做同样的 80%/20% 切分，得到内层训练集与验证集（16,209 条）。双偏差、UserCF、ItemCF 和混合系数都只在内层验证集上选择，外层测试集只在最终评估时使用。调参网格为：

- 双偏差：$\lambda_u \in \{0, 2, 5, 10, 20, 50\}$，$\lambda_i \in \{0, 5, 10, 25, 50, 100\}$；
- UserCF：$K \in \{20, 40, 80, 160, 320, 500\}$，$c_{\min} \in \{3, 5, 10\}$，$\beta_s \in \{0, 5, 15\}$，$\alpha_u \in \{0, 0.2, \ldots, 1.0\}$；
- ItemCF：$K \in \{20, 50, 100\}$，$c_{\min} \in \{1, 2, 3\}$，$\beta_s \in \{5, 15\}$，$\alpha_i \in \{0, 0.25, \ldots, 1.0\}$；
- 混合：$\gamma \in \{0, 0.25, 0.5, 0.75, 1.0, 1.25\}$。

评价指标为测试集 RMSE 与 MAE。另以用户为聚类单位做 2,000 次配对 bootstrap，报告“基线误差减去模型误差”的 95% 区间；区间下界大于 0 才说明改进在用户层面稳定。

3.2 主结果与对照/消融

表 2 汇总测试集结果。空白对照使用全局均值；阴性对照在保持每个用户邻居权重分布与度的前提下随机置换邻居身份，重复 25 次；阳性对照使用一个 3×3 的合成残差矩阵，基线分 3.0，注入同向邻居信号后预测 5.0，增益 2.0，判定阈值为“高于基线 1.0 以上”。

主结果可以拆成三层。第一，双偏差相对用户均分基线把 RMSE 从 0.9750 降至 0.9072，降幅 6.96%；MAE 从 0.7477 降至 0.6949，降幅 7.07%。这说明先扣除大众口碑与个人打分松紧度，能显著减轻协同项拟合一阶统计偏差的负担。第二，在双偏差上单独加入 UserCF 或 ItemCF，RMSE 分别降至 0.9016（相对双偏差降 0.62%）和 0.9015（降 0.63%）；两条通道同时加入后为 0.8979，相对双偏差降 1.03%，相对用户均分基线累计降 7.91%，MAE 累计降 8.37%。第三，减法消融显示两条通道都有贡献：从混合模型中去掉 UserCF 或 ItemCF，RMSE 分别回升到 0.9018 和 0.9024；两项都去掉则回到 0.9072。

Table 2: 测试集指标与对照/消融结果（20,256 条留出评分）

实验类型	模型配置	RMSE	MAE	说明
空白对照	全局均值 μ	1.0647	0.8423	不含任何个性化项。
基线	用户历史均分 $\bar{r}_u$	0.9750	0.7477	只消除用户打分松紧差异。
基线	双偏差 $\mu + b_u + b_i$	0.9072	0.6949	同时剥离用户与电影主效应。
加法	双偏差 + UserCF	0.9016	0.6887	用户残差通道，$\alpha_u = 0.2$。
互补	双偏差 + ItemCF	0.9015	0.6888	物品残差通道，$\alpha_i = 0.25$。
加法	双偏差 + UserCF + ItemCF	0.8979	0.6851	双通道混合，$\gamma = 0.75$。
减法	混合模型去掉 UserCF	0.9018	0.6895	仅保留 ItemCF 修正。
减法	混合模型去掉 ItemCF	0.9024	0.6898	仅保留 UserCF 修正。
减法	混合模型去掉两项	0.9072	0.6949	退回双偏差基线。
阴性对照	随机邻居（25 次均值）	0.9091	0.6958	随机化邻居身份，保留权重分布与度。
阳性对照	合成 3×3 残差	3.0→5.0	—	增益 2.0，通过（阈值 1.0）。

配对 bootstrap 的 95% 区间均不含 0：UserCF 相对双偏差的 ΔRMSE 为 $[0.0037, 0.0076]$，ItemCF 为 $[0.0035, 0.0080]$，混合模型为 $[0.0067, 0.0122]$。阴性对照的 25 次随机邻居平均 RMSE 为 0.9091（5%–95% 分位 0.9082–0.9101），比双偏差的 0.9072 更差，也比真实 UserCF 的 0.9016 差 0.0075 左右；随机化邻居身份后协同增益消失，说明改进并非来自代码漏洞或评分分布偏置。阳性对照能从 3.0 提升到 5.0，说明在强同向信号存在时残差检索能锁定同簇邻居。

需要说明的是，混合模型相对双偏差的绝对提升只有 0.0093 RMSE，属于边际改进；因此报告同时给出效应量、bootstrap 区间和阴性/阳性对照，而不把 0.8979 单独解释为“模型显著更优”。此外，当前指标是评分预测误差，不是 Top-K 排序指标；后者需要 Recall@K、NDCG@K 等度量，见第 5 节。

4 工程实现与交互界面

4.1 模块划分与向量化实现

项目由三个 Python 模块组成：`recommender.py` 负责数据读取、时序切分、双偏差/相似度/残差修正、调参、对照实验和缓存构建；`tui.py` 负责全屏终端界面与单用户文本输出；`main.py` 提供 `--once`、`--evaluate`、`--rebuild`、`--user` 等命令行入口。启动演示时直接读取 `data/ml-latest-small/demo_data.json`，不需要联网或重新训练。

底层以 NumPy 向量化为主：评分矩阵为 float32 的 $671 \times 9{,}066$ 数组，缺失位置是 `NaN`；M 为布尔观测掩码。用户共同评分数通过 $C = MM^{\mathsf{T}}$ 得到，残差内积通过 $E_{\text{dot}} = R_{\text{res}} R_{\text{res}}^{\mathsf{T}}$ 得到；ItemCF 先用 scikit-learn 的余弦最近邻为每部电影取 128 个候选，再只对候选对统计共同评分人数，最后用 CSR 稀疏矩阵保存 Top-K 有向物品邻居。这样既避免 $9{,}066^2$ 的稠密物品相似度矩阵，也避免把缺失值当成 0 分。

内存方面，原始 ID 直接建列的 float64 矩阵约 880.08 MB；本项目的 float32 密集评分矩阵约 24.33 MB，残差 CSR 约 0.80 MB（100,004 个非零元），布尔观测掩码约 0.50 MB。缓存 `demo_data.json` 保存数据集计数、选中参数、测试指标以及每位用户的 Top-50 未看电影推荐。

4.2 TUI 布局与交互

交互界面使用 prompt_toolkit（不是 curses）实现全屏应用，依赖已写入 `requirements.txt`。界面包含四个区域：

1. 顶部信息区：显示评分总数、用户数、有评分电影数，以及当前用户的评分条数、平均分和高分电影摘要。
2. 检索控制栏：三个文本框分别接收用户 ID（Enter 校验并切换）、流派子串（大小写不敏感）和片名子串（大小写不敏感）。
3. 推荐表格与分页器：每页 8 条，列为排名、片名、流派、预测分和支持权重；从该用户的 Top-50 缓存中筛选，已评分电影不会出现。
4. 底部状态与快捷键栏：显示筛选结果数、当前页范围和 F2 上页、F3 下页、Tab/Shift-Tab 切换输入框、Ctrl-Q/Ctrl-C 退出；界面启用鼠标支持。

底部还会显示 UserCF 邻居数、ItemCF 邻居数、混合系数，以及混合模型在测试集上的 RMSE/MAE。需要区分的是，表格中的“支持权重”是 UserCF 与 ItemCF 邻居权重之和，不是残差贡献；二者都在缓存构建时算好。

4.3 用户 66 的推荐案例

用户 66 在数据中有 49 条评分，平均分 3.959。缓存中的前四名及分解如表 3 所示；表中 Δ 为已经乘过混合系数 $\gamma = 0.75$ 的残差贡献，因此“基础分 + 两列贡献”等于预测分（四舍五入误差除外）。《马耳他之鹰》的基础分 4.482 并非最高，ItemCF 贡献 +0.201 使其升至 4.692；在这四条中《肖申克的救赎》基础分 4.632 最高，但 ItemCF 贡献为负（-0.052），退居第四。这个案例说明榜单由基础分与两条残差通道共同决定，而不是简单按大众口碑排序；它只是单用户示例，不能替代总体指标。

Table 3: 用户 66 的前四条缓存推荐及分量分解

排名	电影	预测分	基础分	UserCF Δ	ItemCF Δ
1	Maltese Falcon, The (1941)	4.692	4.482	+0.008	+0.201
2	Godfather: Part II, The (1974)	4.640	4.551	+0.020	+0.069
3	It Happened One Night (1934)	4.625	4.483	+0.044	+0.099
4	Shawshank Redemption, The (1994)	4.592	4.632	+0.012	-0.052

5 结论、局限与改进方向

本实验在 MovieLens ml-latest-small 上完成了紧凑索引、正则化双偏差、残差 UserCF/ItemCF 混合和离线对照评估。原始 ID 直接建列的 float64 矩阵约 880.08 MB，改为 float32 连续索引后约 24.33 MB；测试集 RMSE 从用户均分基线的 0.9750 降至双偏差的 0.9072，最终混合模型为 0.8979，MAE 从 0.7477 降至 0.6851。配对 bootstrap 区间、随机邻居阴性对照和合成信号阳性对照均支持残差协同的边际增益，但绝对提升有限，不能夸大为大幅超越基线。

局限主要有四点：一是只评估评分预测（RMSE/MAE），没有评估 Top-K 排序质量；二是只使用一次外层时间切分，未做重复切分或多折时间评估；三是 ItemCF 用 128 个候选做预筛选，可能漏掉长尾物品邻居；四是没有显式建模时间衰减、内容特征和冷启动用户。后续可以补充 Recall@K、NDCG@K 与覆盖率，做多次时间切分和按时间的衰减因子，并比较更简单的强基线（如全局/用户/物品偏差组合、ItemKNN）在相同协议下的表现。

参考文献与数据来源

1. F. Maxwell Harper and Joseph A. Konstan. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems, 5(4), 2015.
2. GroupLens Research. MovieLens Latest Small Dataset. https://grouplens.org/datasets/movielens/.
3. Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. In Proceedings of ACM RecSys ' 19, 2019. https://arxiv.org/abs/1907.06902.
4. Alan Said and Alejandro Bellogín. Reproducibility in Recommender Systems: A Survey. arXiv:2607.26074, 2026. https://arxiv.org/abs/2607.26074.
5. Alan Said, Maria Soledad Pera, and Michael D. Ekstrand. We' re Still Doing It (All) Wrong: Recommender Systems, Fifteen Years Later. arXiv:2509.09414, 2025. https://arxiv.org/abs/2509.09414.
6. Robin Verachtert, Lien Michiels, and Bart Goethals. Are We Forgetting Something? Correctly Evaluate a Recommender System With an Optimal Training Window. In PERSPECTIVES 2022, CEUR Workshop Proceedings, Vol. 3228, 2022. https://ceur-ws.org/Vol-3228/paper1.pdf.
7. Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-Based Collaborative Filtering Recommendation Algorithms. In Proceedings of WWW ' 01, 2001.
8. Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix Factorization Techniques for Recommender Systems. IEEE Computer, 42(8), 2009.

附录：核心实现代码节选

以下代码直接取自仓库 `recommender.py`，覆盖时序切分、紧凑索引、双偏差估计、收缩残差相似度、UserCF 修正和阳性对照。完整实现、TUI 与评估脚本见仓库根目录。

```python
def split_by_user_time(data: pd.DataFrame, test_fraction: float = 0.2):
    """Hold out each user's latest ratings while keeping at least one training row."""
    train_idx, heldout_idx = [], []
    for _, group in data.groupby("userId", sort=True):
        ordered = group.sort_values(["timestamp", "movieId"], kind="stable")
        indices = ordered.index.to_numpy()
        heldout_count = min(
            len(indices) - 1,
            max(1, int(np.ceil(len(indices) * test_fraction))),
        )
        train_idx.extend(indices[:-heldout_count])
        heldout_idx.extend(indices[-heldout_count:])
    return (
        data.loc[train_idx].reset_index(drop=True),
        data.loc[heldout_idx].reset_index(drop=True),
    )


def make_rating_matrix(data, user_ids, movie_ids):
    """Use contiguous positions; NaN is missing and never treated as a zero rating."""
    user_pos = pd.Index(user_ids).get_indexer(data["userId"])
    movie_pos = pd.Index(movie_ids).get_indexer(data["movieId"])
    if (user_pos < 0).any() or (movie_pos < 0).any():
        raise ValueError("Input contains a userId or movieId outside the index maps.")
    matrix = np.full((len(user_ids), len(movie_ids)), np.nan, dtype=np.float32)
    matrix[user_pos, movie_pos] = data["rating"].to_numpy(dtype=np.float32)
    return matrix
```

代码 1: 时序切分与连续索引评分矩阵

```python
def _estimate_biases(rows, cols, values, n_users, n_movies, lambda_user, lambda_movie, epochs=10):
    """Alternating ridge-style user and movie bias estimates."""
    global_mean = float(values.mean())
    user_count = np.bincount(rows, minlength=n_users).astype(np.float32)
    movie_count = np.bincount(cols, minlength=n_movies).astype(np.float32)
    user_bias = np.zeros(n_users, dtype=np.float32)
    movie_bias = np.zeros(n_movies, dtype=np.float32)
    for _ in range(epochs):
        user_numerator = np.bincount(
            rows,
            weights=values - global_mean - movie_bias[cols],
            minlength=n_users,
        )
        movie_numerator = np.bincount(
            cols,
            weights=values - global_mean - user_bias[rows],
            minlength=n_movies,
        )
        user_denominator = lambda_user + user_count
        movie_denominator = lambda_movie + movie_count
        user_bias = np.divide(
            user_numerator, user_denominator,
            out=np.zeros(n_users, dtype=np.float32), where=user_denominator > 0,
        )
        movie_bias = np.divide(
            movie_numerator, movie_denominator,
            out=np.zeros(n_movies, dtype=np.float32), where=movie_denominator > 0,
        )
    return global_mean, user_bias, movie_bias
```

代码 2: 交替更新的正则化双偏差

```python
def prepare_user_similarity(fit, k_neighbors, min_common, shrinkage):
    similarity = fit["similarity"].copy()
    overlap = fit["overlap"]
    similarity[overlap < min_common] = 0.0
    if shrinkage > 0:
        similarity *= overlap / (overlap + shrinkage)
    np.maximum(similarity, 0.0, out=similarity)
    np.fill_diagonal(similarity, 0.0)
    for user_row in range(len(similarity)):
        positive = np.flatnonzero(similarity[user_row] > 0)
        if len(positive) > k_neighbors:
            keep = positive[np.argpartition(similarity[user_row, positive], -k_neighbors)[-k_neighbors:]]
            keep_mask = np.zeros(len(similarity), dtype=bool)
            keep_mask[keep] = True
            similarity[user_row, ~keep_mask] = 0.0
    return similarity

```

```python

def residual_correction(fit, k_neighbors, min_common, shrinkage):
    """Compute top-K positive user-neighbor residual corrections."""
    similarity = prepare_user_similarity(fit, k_neighbors, min_common, shrinkage)
    weights = similarity @ fit["observed"].astype(np.float32)
    weighted_residual = similarity @ fit["residual"]
    correction = np.divide(
        weighted_residual,
        weights,
        out=np.zeros_like(weighted_residual),
        where=weights > 0,
    )
    return correction, weights
```

代码 3: 收缩残差相似度与 UserCF 修正

```python
def synthetic_positive_control():
    """Check that a planted similar-user signal moves a withheld prediction upward."""
    residual = np.array([
        [1.0, 1.0, 0.0],
        [1.0, 1.0, 2.0],
        [-1.0, -1.0, -2.0],
    ], dtype=np.float32)
    observed = np.array([
        [True, True, False],
        [True, True, True],
        [True, True, True],
    ])
    norms = np.sqrt(np.sum(residual * residual, axis=1))
    similarity = residual @ residual.T / np.maximum(norms[:, None] * norms[None, :], 1e-12)
    overlap = observed.astype(np.int32) @ observed.astype(np.int32).T
    np.fill_diagonal(similarity, 0.0)
    fit = {
        "similarity": similarity,
        "overlap": overlap,
        "observed": observed,
        "residual": residual,
        "rating_min": 0.5,
        "rating_max": 5.0,
        "baseline": np.full((3, 3), 3.0, dtype=np.float32),
    }
    correction, _ = residual_correction(fit, 2, 2, 0.0)
    prediction = float(np.clip(fit["baseline"][0, 2] + correction[0, 2], 0.5, 5.0))
    baseline = float(fit["baseline"][0, 2])
    return {
        "passed": bool(prediction > baseline + 1.0),
        "baseline": baseline,
        "prediction": prediction,
        "gain": prediction - baseline,
    }
```

代码 4: 合成信号阳性对照